Bellabeat Case Study

Michael Pon

2024-10-06

Introduction

This R Markdown Document contains the data transformation, visualization and analysis of the data provided in the Google Analytics Certificate Case Study (Option 2): Bellabeat.

After initial exploration it was discovered all data had be organized in either a daily or hourly format. I opted to transform and visualize the data independently.

The R Script in the project repository includes ALL code I used to complete this work. As my first foray into R there was quite a bit of tinkering to comprehend how everything works.

Data Citation:
Furberg, R., Brinton, J., Keating, M., & Ortiz, A. (2016). Crowd-sourced Fitbit datasets 03.12.2016-05.12.2016 [Data set]. Zenodo. https://doi.org/10.5281/zenodo.53894

You will find 7 different sections in this document.

1. Package Installation
- This was included to show the packages I utilized for the work.
- Reshape2 was the only package not presented in the course.

2. Hourly Data Transformation
- This is the transformation process of the hourly data.

3. Hourly Data Visualization
- This is the visualization process of the hourly data.

4. Daily Data Transformation
- This is the transformation process of the daily data.

5. Daily Data Visualization
- This is the visualization process of the daily data.

6. Analysis Findings
- This is includes commentary about the trend findings in the data.

7. Recommendations
- Here I articulate how I propose the Bellabeat marketing team use my findings.

1. Package Installation

For the start to finish data reading, manipulation, visualization, and analysis I used:

```
library(tidyverse)
library(lubridate)
library(dplyr)
library(reshape2)
```

2. Hourly Data Transformation

All .csv files provided were pulled into environment with read.csv()

```
calories_hourly <- read.csv('hourlyCalories_merged.csv')
intensities_hourly <- read.csv('hourlyIntensities_merged.csv')
steps_hourly <- read.csv('hourlySteps_merged.csv')
```

Verifying no duplication (comparing unique row counts).

```
hourlyrows <- (
  nrow(calories_hourly) +
    nrow(intensities_hourly) +
    nrow(steps_hourly)
  )
hourlyuniquerows <- (
  nrow(unique(calories_hourly)) +
    nrow(unique(intensities_hourly)) +
    nrow(unique(steps_hourly))
  )

#TEST#
if (hourlyrows == hourlyuniquerows){
  print('There are no duplicate rows in the data')
  } else {
  print('Duplicates are present')
  }
```

```
## [1] "There are no duplicate rows in the data"
```

Confirming each unique user is present in each file:

```
n_distinct(calories_hourly$Id)
```

```
## [1] 33
```

```
n_distinct(intensities_hourly$Id)
```

```
## [1] 33
```

```
n_distinct(steps_hourly$Id)
```

```
## [1] 33
```

```
# There are 33 distinct user IDs in each file
```

Confirming that each ID was in every file:

```r
#Make a list of the IDs in each file.
hrID <- (c(unique(calories_hourly$Id),
           unique(intensities_hourly$Id),
           unique(steps_hourly$Id)))
#This data was numeric, I transformed it into a data frame and summarized it
#by the frequency of each ID.
hrIDmatrix <- matrix(data=hrID, ncol=1, byrow = TRUE)
hrIDdf <- as.data.frame(hrIDmatrix)
hrIDsummary <- hrIDdf %>%
  group_by(V1) %>%
  summarise(frequency = n())

#I then tested this by testing whether the unique frequency was "==3"
if (unique(hrIDsummary$frequency) == 3){
  print('Each unique user is present in each dataset')
} else {
  print('Each unique user is NOT present in each dataset')
}
```

```
## [1] "Each unique user is present in each dataset"
```

```r
# Great! Only one unique value of 3, there are 33 unique users and 99 total observations.
```

Date Conversions For Aggregation
Data wouldn't merge with the existing date formats.

```r
data.class(calories_hourly$ActivityHour)
```

```
## [1] "character"
```

```r
data.class(intensities_hourly$ActivityHour)
```

```
## [1] "character"
```

```r
data.class(steps_hourly$ActivityHour)
```

```
## [1] "character"
```

```r
#Date data provided were all characters.

#Conversion to date/time (thanks Lubridate!).
calories_hourly$dateclean <- mdy_hms(calories_hourly$ActivityHour)
intensities_hourly$dateclean <- mdy_hms(intensities_hourly$ActivityHour)
steps_hourly$dateclean <- mdy_hms(steps_hourly$ActivityHour)

#mutated the date data to hour values for visualizations.
calories_hourly <- calories_hourly %>%
  mutate(hour = hour(calories_hourly$dateclean))
intensities_hourly <- intensities_hourly %>%
  mutate(hour = hour(intensities_hourly$dateclean))
steps_hourly <- steps_hourly %>%
  mutate(hour = hour(steps_hourly$dateclean))
```

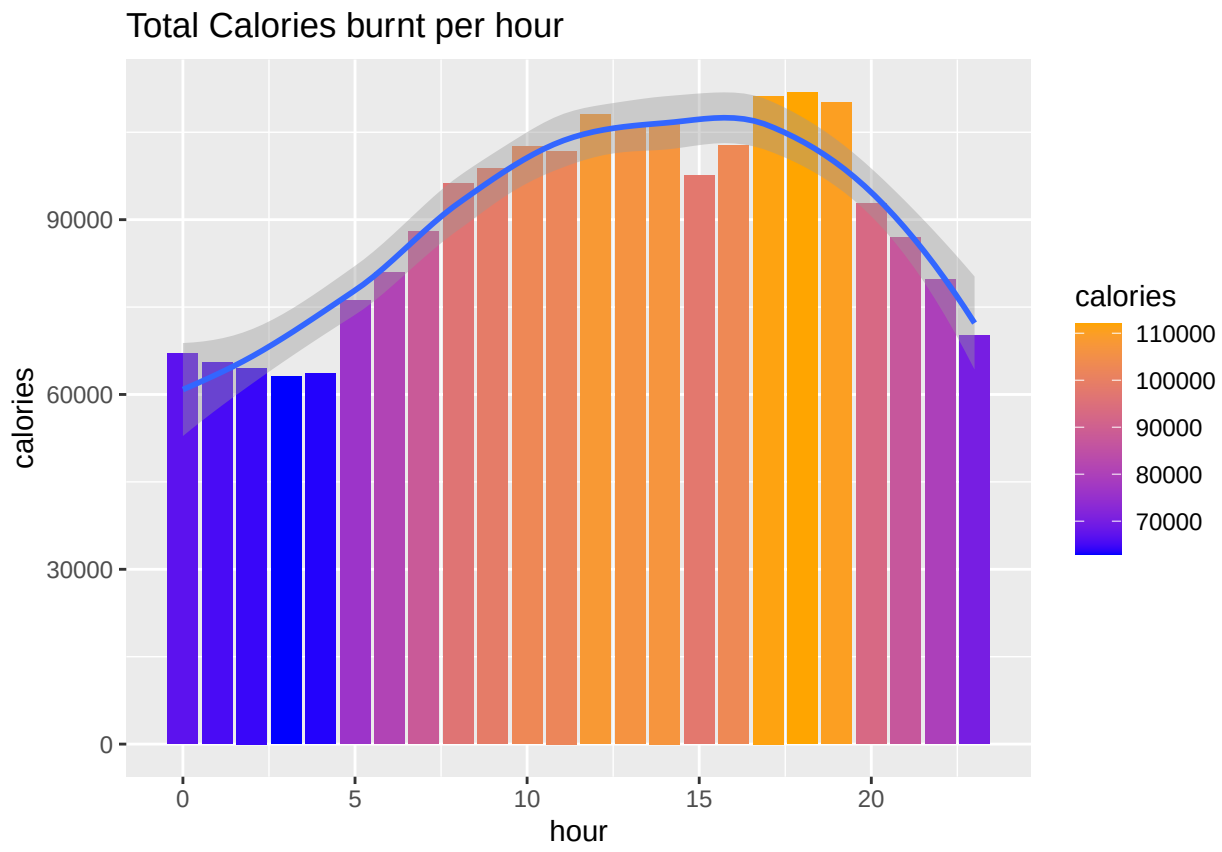
Once data had a consistent format, I aggregated the tables individually by ID/hour for visualizing.

```
calories_aggregated <- aggregate(x = calories_hourly$Calories,
                                 by=list(calories_hourly$hour),
                                 FUN = sum)
colnames(calories_aggregated) <- c('hour', 'calories')

intensities_aggregated <- aggregate(x=intensities_hourly$TotalIntensity,
                                    by=list(intensities_hourly$hour),
                                    FUN = sum)
colnames(intensities_aggregated) <- c('hour', 'intenseminutes')

steps_aggregated <- aggregate(x = steps_hourly$StepTotal,
                              by=list(steps_hourly$hour),
                              FUN = sum)
colnames(steps_aggregated) <- c('hour', 'steptotal')
```

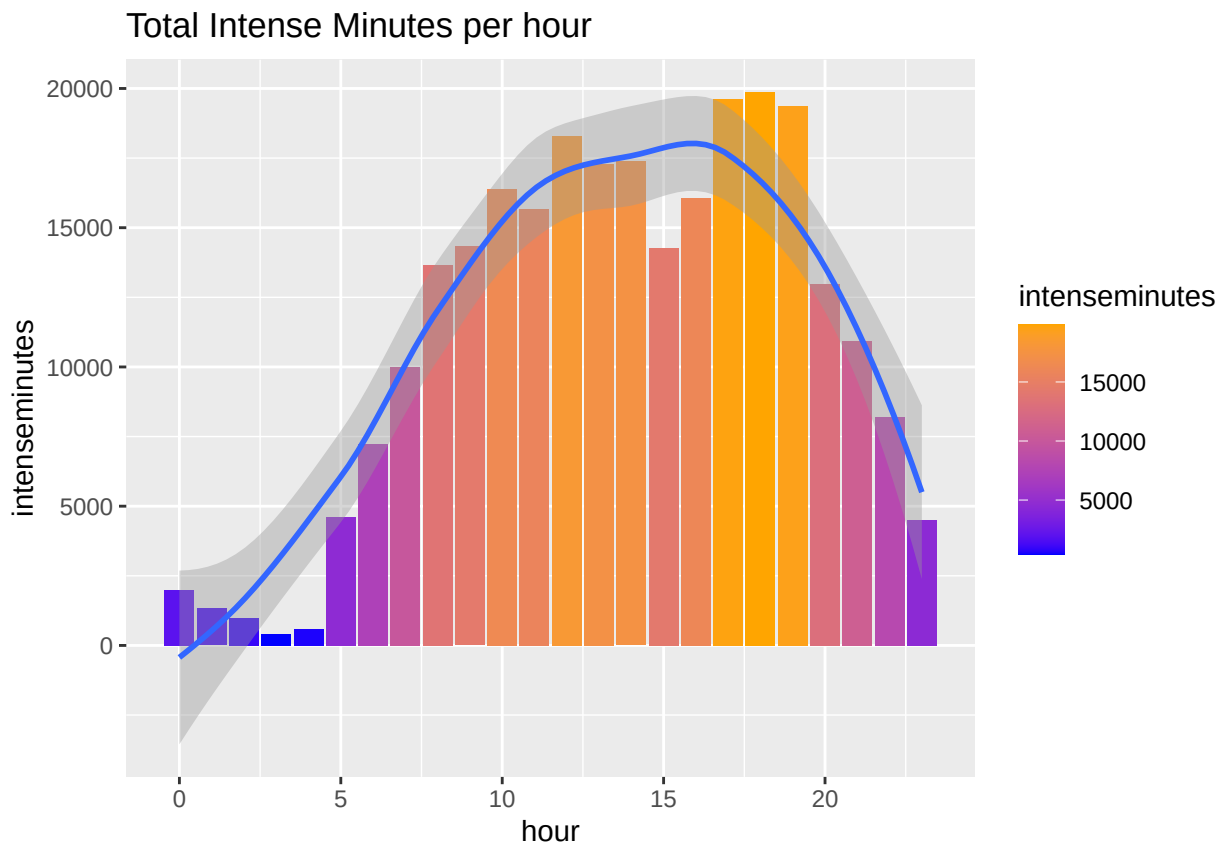
3. Hourly Data Visualization

With the data aggregated, I opted to look for some basic trends in the tables independently.

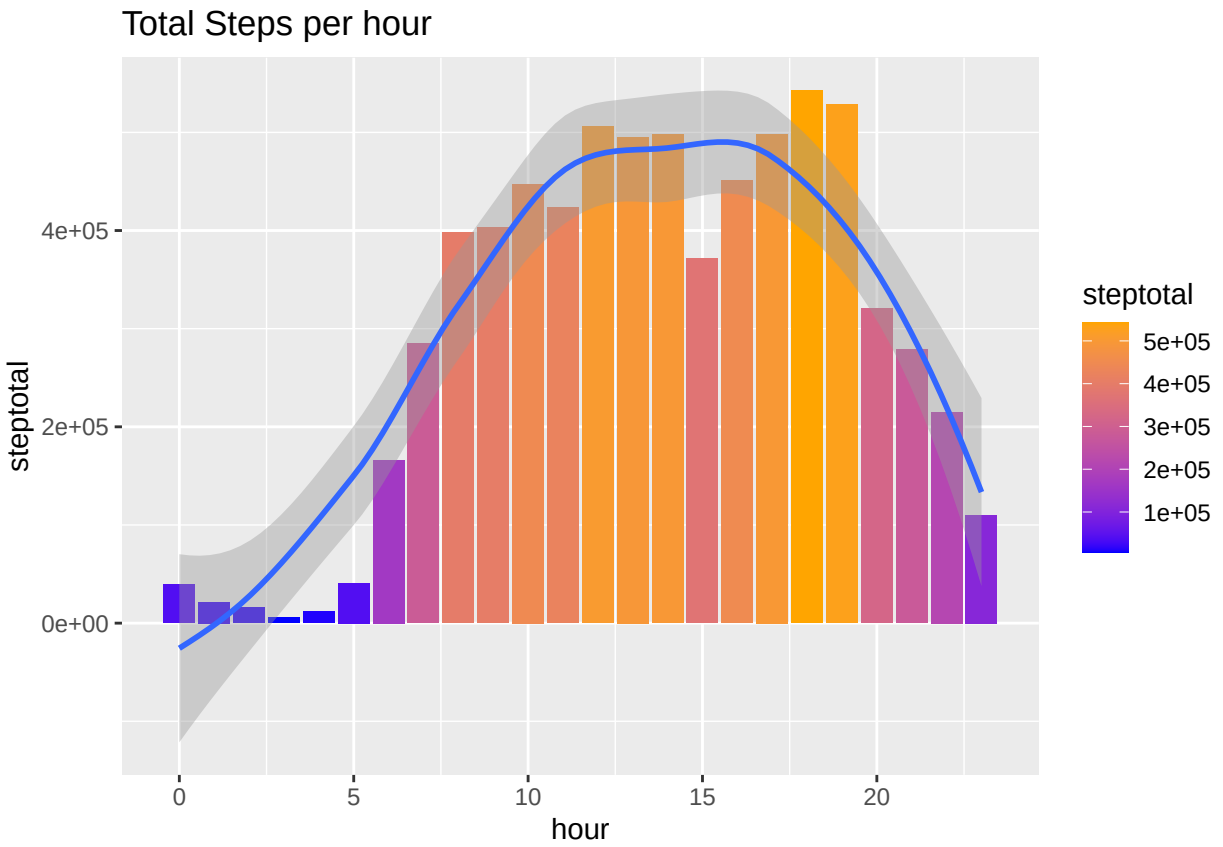
```
#Calories -> users burn the most calories in the afternoon (noon - 6pm).
#There was a clear decline in the 3pm-4pm hour.
ggplot(data = calories_aggregated,
       aes(x = hour,
           y = calories,
           fill = calories)) +
  geom_bar(stat='identity', na.rm=TRUE) +
  scale_fill_gradient(low="blue", high="orange") +
  geom_smooth() +
  labs(title = "Total Calories burnt per hour")
```




```
#Intensities -> users are most intense during a similar period to calories, activity falls at 3pm.
ggplot(data = intensities_aggregated,
       aes(x = hour,
           y = intenseminutes,
           fill = intenseminutes)) +
  geom_bar(stat='identity', na.rm=FALSE) +
  scale_fill_gradient(low="blue", high="orange") +
  geom_smooth() +
    labs(title = "Total Intense Minutes per hour")
```




```
#Steps -> trends align with previous charts. this makes sense!
ggplot(data = steps_aggregated,
       aes(x=hour,
           y=steptotal,
           fill=steptotal)) +
  geom_bar(stat='identity', na.rm=TRUE) +
  scale_fill_gradient(low="blue", high="orange") +
  geom_smooth() +
  labs(title = "Total Steps per hour")
```



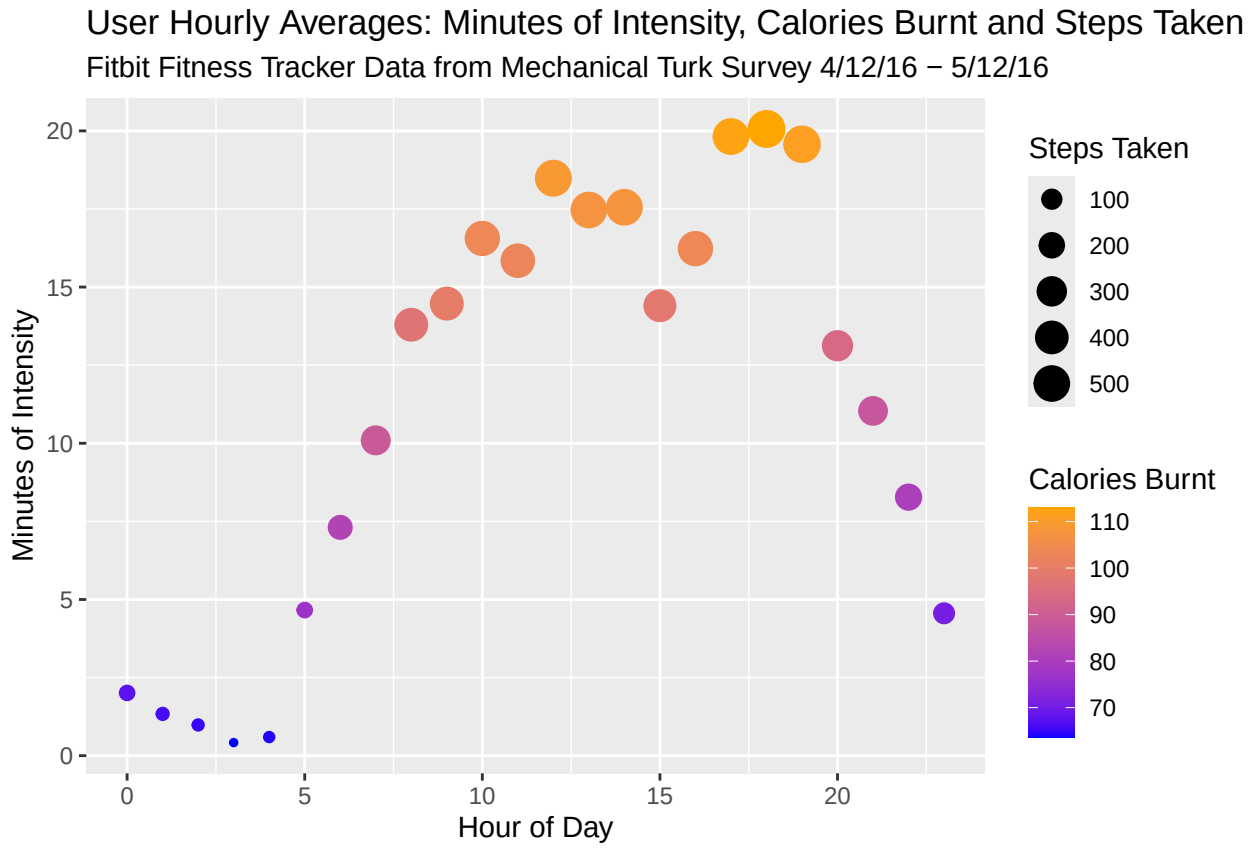
To clearly show the consistent relationship between the three, I opted to create an hourly visualization that included the metrics from each table.

```
#Joining the aggregated tables
hourlyjoin <- calories_aggregated %>%
  left_join(steps_aggregated, by = 'hour') %>%
  left_join(intensities_aggregated, by = 'hour')

#Day count in dataset.
d1 = as_date('2016-04-12')
d2 = as_date('2016-05-12')
difftime(d2, d1, 'days')
```

```
## Time difference of 30 days
```

```
#Visualize!
#Note: the dividing of data points by 33 and 30 is to get the user daily averages (33 users/30 days wit
ggplot(hourlyjoin,
       aes(x = hour,
           y = intenseminutes/33/30)) +
  geom_point(aes(color = calories/33/30,
                 size = steptotal/33/30)) +
  scale_color_gradient(low="blue",
                       high="orange") +
  labs(title = 'User Hourly Averages: Minutes of Intensity, Calories Burnt and Steps Taken',
       subtitle = 'Fitbit Fitness Tracker Data from Mechanical Turk Survey 4/12/16 - 5/12/16',
```

```
      y = 'Minutes of Intensity',
      x = 'Hour of Day',
      color = 'Calories Burnt',
      size = 'Steps Taken')
```



4. Daily Data Transformation

Bringing the data into the environment:

```
sleep_daily <- read.csv('sleepDay_merged.csv')
activity_daily <- read.csv('dailyActivity_merged.csv')
calories_daily <- read.csv('dailyCalories_merged.csv')
intensity_daily <- read.csv('dailyIntensities_merged.csv')
steps_daily <- read.csv('dailySteps_merged.csv')
```

Verified no duplication in the data.
This was similar strategy to what I did with the hourly data.

```
dailyrows <- (
  nrow(sleep_daily) +
    nrow(activity_daily) +
    nrow(calories_daily)+
    nrow(intensity_daily)+
    nrow(steps_daily)
)
dailyuniquerows <- (
  nrow(unique(sleep_daily)) +
    nrow(unique(activity_daily)) +
    nrow(unique(calories_daily))+
    nrow(unique(intensity_daily))+
    nrow(unique(steps_daily))
)

#TEST#
if (dailyrows == dailyuniquerows){
  print('There are no duplicate rows in the data')
} else {
  print('Duplicates are present')
}
```

```
## [1] "Duplicates are present"
```

```
#There were a few duplicate rows, I overwrote the data frames using their
#unique rows (probably not best practice)
sleep_daily <- unique(sleep_daily)
activity_daily <- unique(activity_daily)
calories_daily <- unique(calories_daily)
intensity_daily <- unique(intensity_daily)
steps_daily <- unique(steps_daily)

#Run test again
dailyrows <- (
  nrow(sleep_daily) +
    nrow(activity_daily) +
    nrow(calories_daily)+
    nrow(intensity_daily)+
    nrow(steps_daily)
)
```

```
dailyuniquerows <- (
  nrow(unique(sleep_daily)) +
    nrow(unique(activity_daily)) +
    nrow(unique(calories_daily))+
    nrow(unique(intensity_daily))+
    nrow(unique(steps_daily))
)

#TEST#
if (dailyrows == dailyuniquerows){
  print('There are no duplicate rows in the data')
} else {
  print('Duplicates are present')
}
```

```
## [1] "There are no duplicate rows in the data"
```

```
#No more dupes!
```

Confirming each unique user is present in each table. *Similar strategy again to the hourly data analysis*

```
n_distinct(sleep_daily$Id)
```

```
## [1] 24
```

```
n_distinct(activity_daily$Id)
```

```
## [1] 33
```

```
n_distinct(calories_daily$Id)
```

```
## [1] 33
```

```
n_distinct(intensity_daily$Id)
```

```
## [1] 33
```

```
n_distinct(steps_daily$Id)
```

```
## [1] 33
```

```
#33 users are present except for in sleep daily (24)

#list of the IDs in the file.
dayID <- c(unique(sleep_daily$Id),
           unique(activity_daily$Id),
           unique(calories_daily$Id),
           unique(intensity_daily$Id),
```

```
          unique(steps_daily$Id))

#transforming numeric data to matrix and then dataframe
#one column IDs, fill by row!
dayIDmatrix <- matrix(data=dayID, ncol=1, byrow = TRUE)
dayIDdf <- as.data.frame(dayIDmatrix)

dayIDsummary <- dayIDdf %>%
  group_by(V1) %>%
  summarise(frequency = n())

dayIDsummary
```

```
## # A tibble: 33 x 2
##            V1 frequency
##         <dbl>     <int>
##  1 1503960366         5
##  2 1624580081         4
##  3 1644430081         5
##  4 1844505072         5
##  5 1927972279         5
##  6 2022484408         4
##  7 2026352035         5
##  8 2320127002         5
##  9 2347167796         5
## 10 2873212765         4
## # i 23 more rows
```

```
sum(dayIDsummary$frequency)
```

```
## [1] 156
```

```
#How many Id's am I expecting? 156.
#4 tables with 33 users, 1 with 24.
4*33+24
```

```
## [1] 156
```

```
#The ID frequency mean should be 156/33 if each user is present in each table.

#TEST#
if (mean(dayIDsummary$frequency) == 156/33){
  print('Each unique user is present in each dataset')
} else {
  print('Each unique user is NOT present in each dataset')
}
```

```
## [1] "Each unique user is present in each dataset"
```

Data conversions for aggregating and joining.
Ran into a similar issue as the hourly data, all the dates were characters

```
data.class(sleep_daily$SleepDay)
```

```
## [1] "character"
```

```
data.class(activity_daily$ActivityDate)
```

```
## [1] "character"
```

```
data.class(calories_daily$ActivityDay)
```

```
## [1] "character"
```

```
data.class(intensity_daily$ActivityDay)
```

```
## [1] "character"
```

```
data.class(steps_daily$ActivityDay)
```

```
## [1] "character"
```

```
#all characters, let's add a date column.

#convert to date (thanks again Lubridate)
sleep_daily$date <- mdy_hms(sleep_daily$SleepDay)
activity_daily$date <- mdy(activity_daily$ActivityDate)
calories_daily$date <- mdy(calories_daily$ActivityDay)
intensity_daily$date <- mdy(intensity_daily$ActivityDay)
steps_daily$date <- mdy(steps_daily$ActivityDay)
```

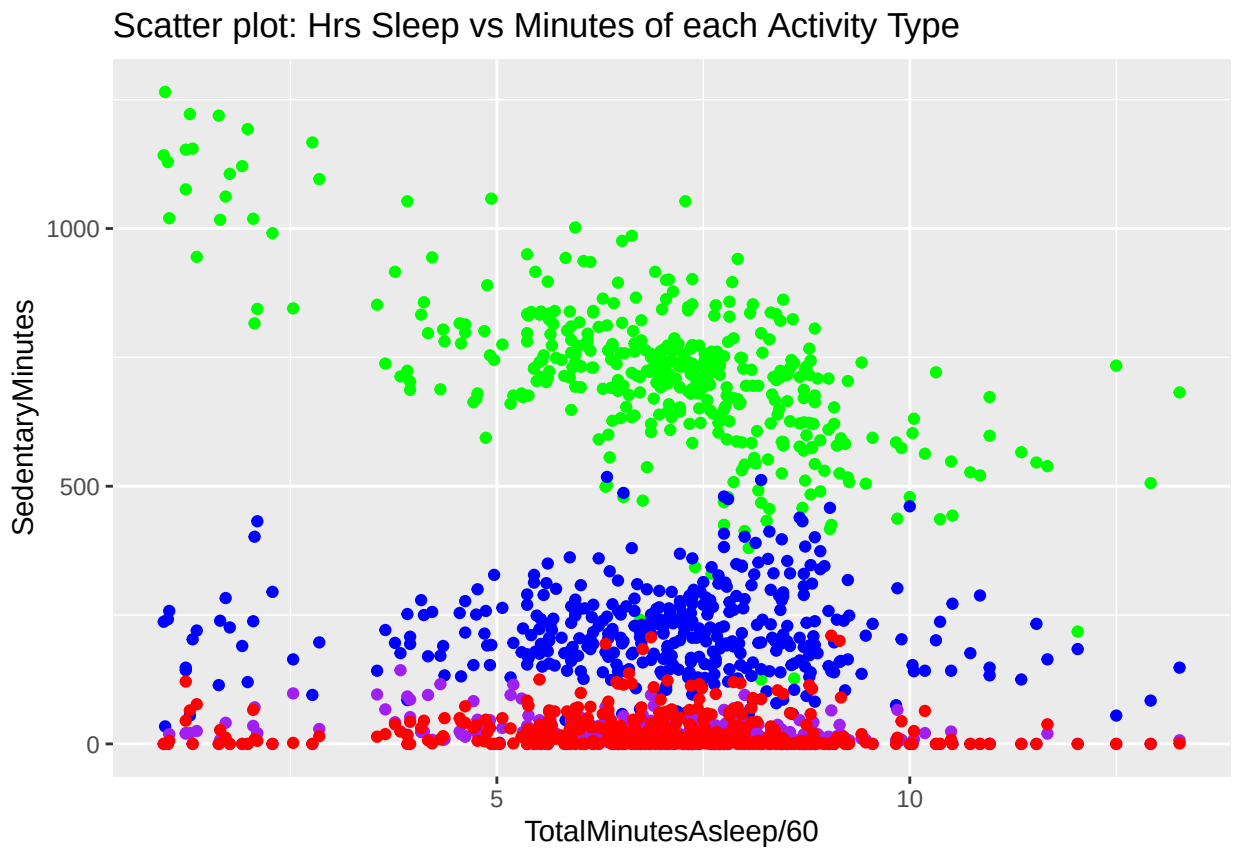
Joining all the daily tables for visualization.

```
dailydata <- left_join(sleep_daily, activity_daily, by = c('Id', 'date'))

dailydata <-left_join(dailydata, calories_daily, by = c('Id', 'date')) %>%
left_join(dailydata, intensity_daily, by = c('Id', 'date')) %>%
left_join(dailydata, steps_daily, by = c('Id', 'date'))

#All the columns joined got pretty messy, I wanted just a select few.
dailydataclean <- dailydata[,c(1,6,20,21,41:57)]
```

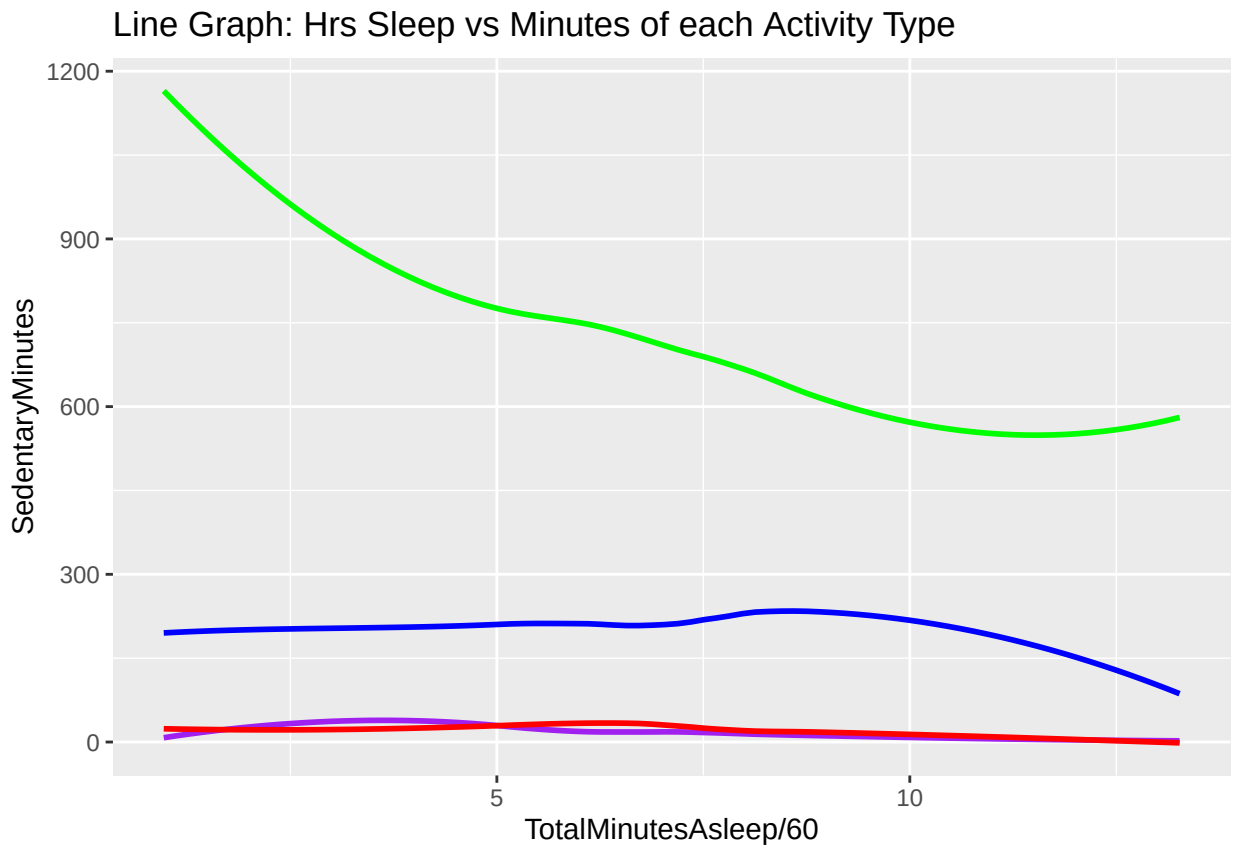
5. Daily Data Visualization

Visualizing the data independently to identify some basic trends.

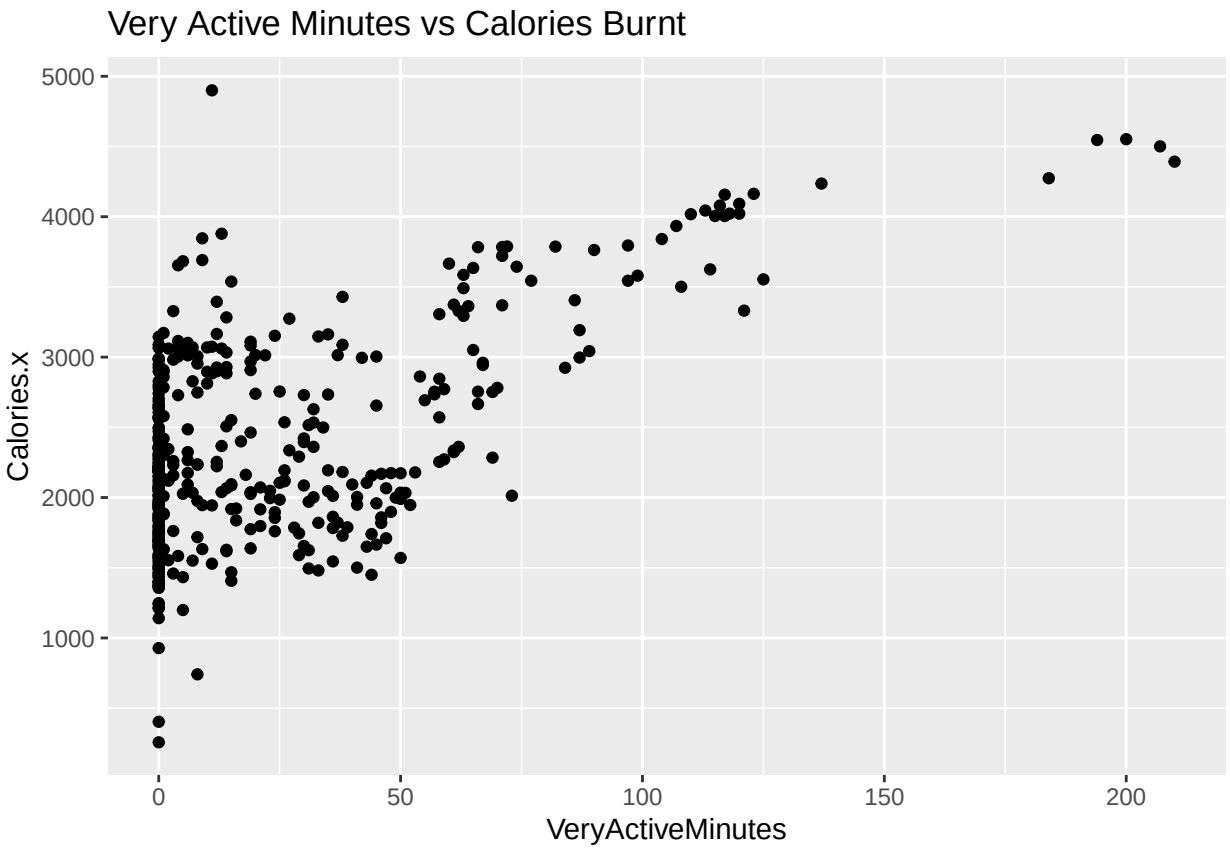
```
activityplot <- ggplot(data = dailydataclean)

activityplot +
  geom_point(data=dailydataclean,
             aes(x=TotalMinutesAsleep/60,
                 y=SedentaryMinutes),
             color = 'green') +
  geom_point(data=dailydataclean,
             aes(x=TotalMinutesAsleep/60,
                 y=LightlyActiveMinutes),
             color = 'blue') +
  geom_point(data=dailydataclean,
             aes(x=TotalMinutesAsleep/60,
                 y=FairlyActiveMinutes),
             color ='purple') +
  geom_point(data=dailydataclean,
             aes(x=TotalMinutesAsleep/60,
                 y=VeryActiveMinutes),
             color ='red') +
  labs(title= 'Scatter plot: Hrs Sleep vs Minutes of each Activity Type')
```



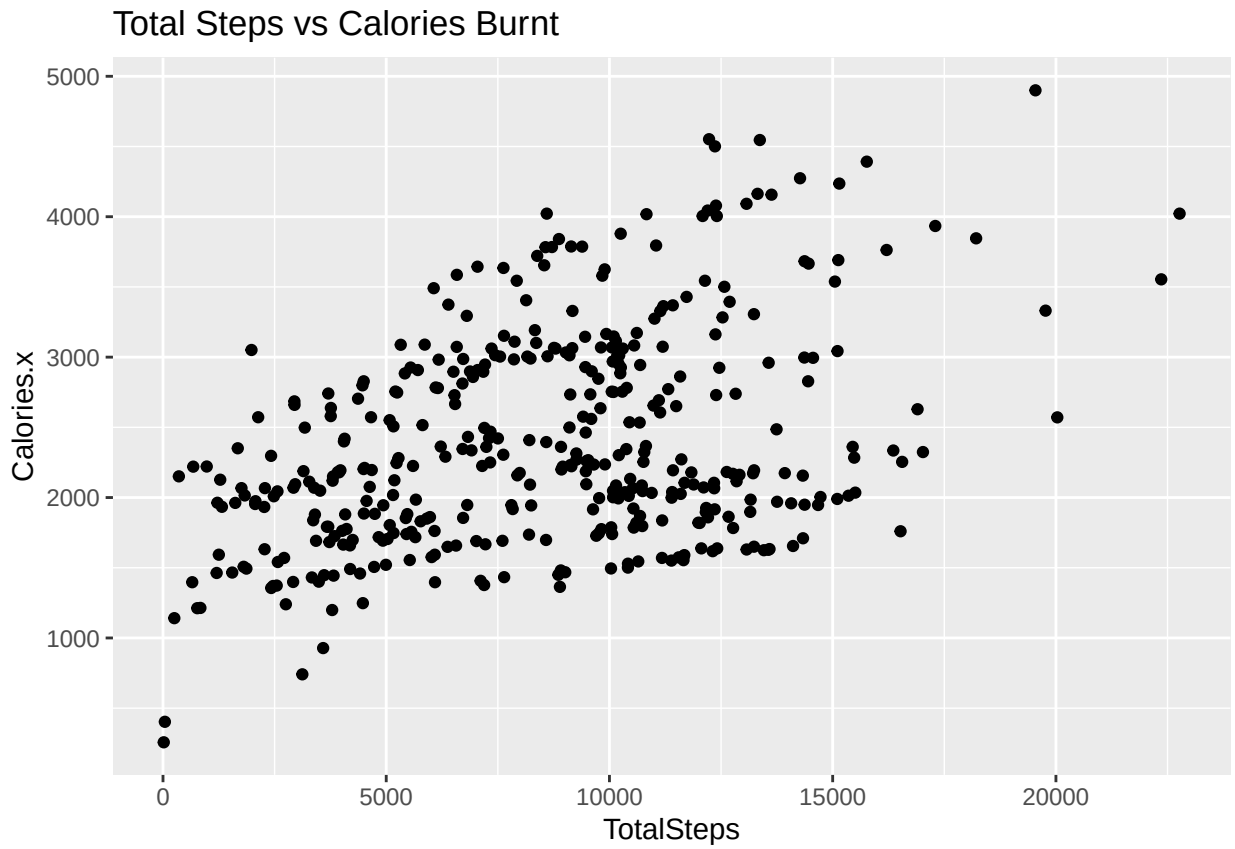
```
activityplot +
  geom_smooth(data=dailydataclean,
              aes(x=TotalMinutesAsleep/60,
                  y=SedentaryMinutes), se = FALSE, color = 'green') +
  geom_smooth(data=dailydataclean,
              aes(x=TotalMinutesAsleep/60,
                  y=LightlyActiveMinutes), se = FALSE, color = 'blue') +
  geom_smooth(data=dailydataclean,
              aes(x=TotalMinutesAsleep/60,
                  y=FairlyActiveMinutes),
              se = FALSE, color = 'purple') +
  geom_smooth(data=dailydataclean,
              aes(x=TotalMinutesAsleep/60,
                  y=VeryActiveMinutes), se = FALSE, color = 'red') +
  labs(title= 'Line Graph: Hrs Sleep vs Minutes of each Activity Type')
```




```
#Key activity and sleep trend -> sedentary minutes down with increased sleep hours.
#Faceted graphs would allow for a better analysis here. More to come!

#After analyzing the activity types, I explored teh data a little further.
ggplot(data = dailydataclean) +
  geom_point(aes(x=VeryActiveMinutes, y=Calories.x)) +
  labs(title = 'Very Active Minutes vs Calories Burnt')
```




```
ggplot(data = dailydataclean) +
  geom_point(aes(x=TotalSteps, y=Calories.x)) +
  labs(title = 'Total Steps vs Calories Burnt')
```




```
#More activity minutes/steps more calories burnt. Quite logical.
```

Like hourly, I wanted to join this all together in a single cohesive table. But wanted to look into the relationship of activity types and sleep after my initial observations. It required a "reshape" of the activity data.

```
#Melting data to long format with every activity type is it's own record with ID/date.
activitydailylong <- melt(activity_daily,
                     id.vars = c('Id', 'date'),
                     measure.vars = c('VeryActiveMinutes',
                                      'FairlyActiveMinutes',
                                      'LightlyActiveMinutes',
                                      'SedentaryMinutes'))

#We then joined the long data with the rest of the table
#to match the hours of sleep to each record.
adljoin <- left_join(activitydailylong, dailydataclean, by = c('Id', 'date'))
```

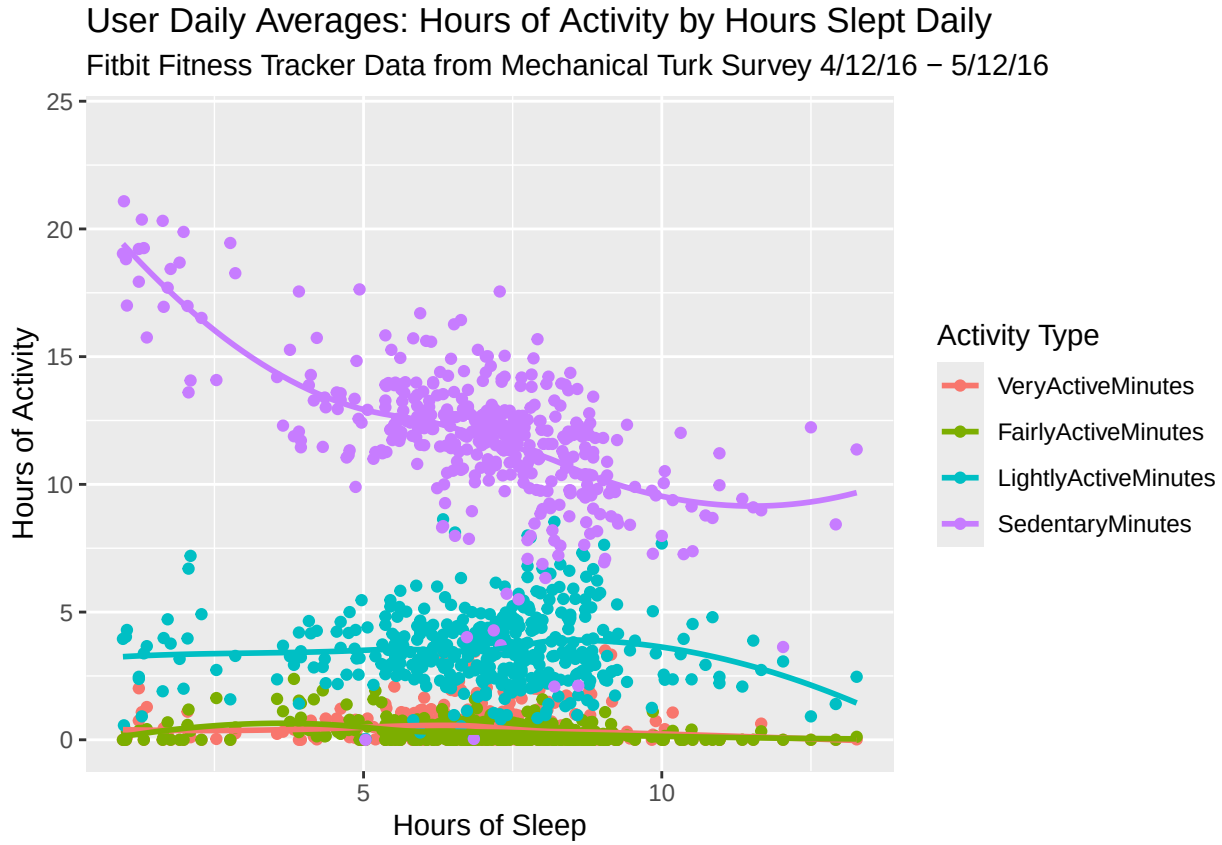
Once data was "melted", I produced a visual that first visualized the hours of sleep vs activity on one table. *melt() admittedly took me a second to wrap my head around*

```
ggplot(data = adljoin, aes(TotalMinutesAsleep/60, value/60, group = variable, color = variable)) +
  geom_point() +
  geom_smooth(se=FALSE) +
  #  stat_summary(fun = 'mean')
```

```
  labs(title = 'User Daily Averages: Hours of Activity by Hours Slept Daily',
       subtitle = 'Fitbit Fitness Tracker Data from Mechanical Turk Survey 4/12/16 - 5/12/16',
       x = 'Hours of Sleep',
       y = 'Hours of Activity',
       color = 'Activity Type')
```

```
## Warning: Removed 2120 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 2120 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



There is a warning message received here about 2,120 records containing non-finite data. This was a result of users not having a full daily set of sleep data.

```
sleepanalysis <- adljoin %>%
  group_by(TotalSleepRecords) %>%
  summarise(n=n())

sleepanalysis
```

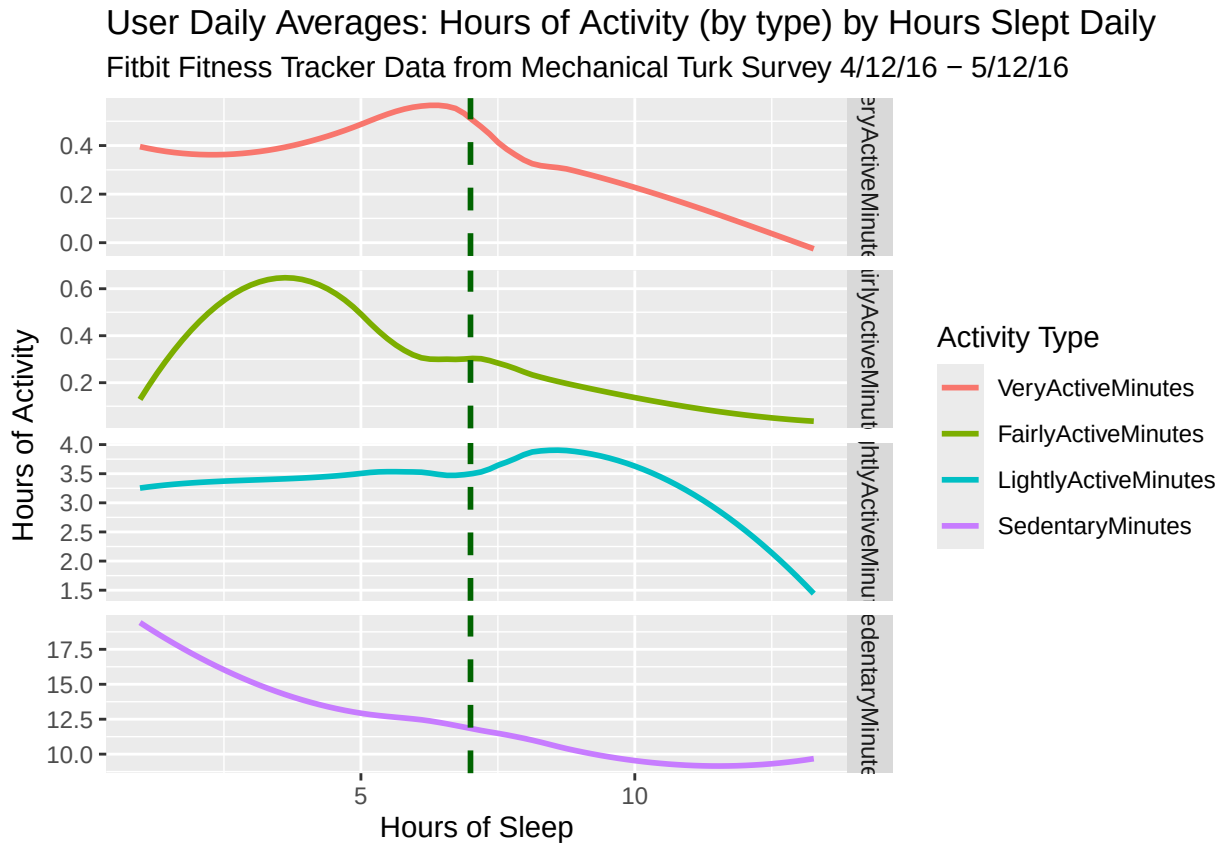
```
## # A tibble: 4 x 2
##   TotalSleepRecords     n
##               <int> <int>
## 1                 1  1456
```

```
## 2                 2   172
## 3                 3    12
## 4                NA  2120
```

```
#The 2,120 rows containing non-finite data are the 2,120 records with a missing
##(NA) entry for TotalSleepRecords.
```

I realized the scaling prevented any meaningful insight into how sleep was impacting the more active variables (which had minute counts FAR less than sedentary).
I opted to used facet_grid to separate them out.

```
ggplot(data = adljoin, aes(TotalMinutesAsleep/60, value/60, group = variable, color = variable)) +
  geom_smooth(se=FALSE) +
  #  stat_summary(fun = 'mean') +
  geom_vline(xintercept = 7, linetype = "dashed", color = " dark green", size = 1) +
  facet_grid(rows= vars(variable), scales = 'free') +
  labs(title = 'User Daily Averages: Hours of Activity (by type) by Hours Slept Daily',
       subtitle =
       'Fitbit Fitness Tracker Data from Mechanical Turk Survey 4/12/16 - 5/12/16',
       x = ('Hours of Sleep'),
       y = 'Hours of Activity',
       color = 'Activity Type')
```



```
#The horizontal line placed at hour 7 displays how a "full nights rest" impacts
#activity.
```

6. Analysis Findings

With the tables visualized, I began analyzing the data with the key question in mind. I was to analyze the non-Bellabeat device user data to inform the marketing strategy to positively impact usage/sales of their devices.

The data provided tables that included human activity data organized in hourly and daily data sets. Because of this, I transformed and visualized the data separately to analyze separately.

Hourly Data Analysis

After transforming the hourly data and visualizing them all together in the "User Hourly Averages" visual, I uncovered a few basic trends:
- Users were **MOST active during the late morning and afternoon hours**.
Activity peaked in the hours generally after work for those working standard business hours (~4pm-7pm).
- User **activity was generally the LEAST during the overnight and late evening hours**.
This is a logical observation but further implies the vast majority of the 33 included users of Fitbit devices are operating on work/life schedules that aren't nocturnal or night shifts.
- User minutes of **activity came down at 8pm (daily hour 20).** *Intense activity minutes didn't fall completely into the "sleep" hour averages (<2.5 minutes of intense activity/hr) until ~midnight (daily hour 0).*

Daily Data Analysis

The completion of the hourly data analysis generally fixated me on the relationship between activity and sleep with the user data we had. The availability of the same data on a daily dataset made it a no-brainer to dive into.

I was very much interested in the relationships, but when the initial visuals didn't allow for a real good look into the trends of activity minutes (by level of intensity) because of the scaling that the visuals were presented with I created the initial "User Daily Averages" table. The second, faceted with facet_grid() allowed for a better analysis.

The horizontal line in the second "User Daily Averages" table was my quick way of visualizing an "average" nights rest.

"Teenagers typically sleep eight hours. 40-year-olds typically sleep about seven hours, and about 6.5 hours of sleep is the average for people between the ages of 55 and 60. But these are all only averages. Everyone needs a different amount of sleep."
InformedHealth.org [Internet]. Cologne, Germany: Institute for Quality and Efficiency in Health Care (IQWiG); 2006-. In brief: What is "normal" sleep? [Updated 2024 Aug 1]. Available from: https://www.ncbi.nlm.nih.gov/books/NBK279322/

With the visual completed, I found some unexpected trends with sleep and activity types.

- **Very Active Hours PEAKED right before the 7 hour sleep mark** and then steadily declined. *My initial thought here was that users that slept more than the average might be more sedentary than their more "sleepless" counterparts.*
- **Fairly Active Hours strangely PEAKED at less than 5 hours of sleep** and then declined continually. *This strange trend didn't make any sense to me at all. My instinct tells me that these fairly active minutes should increase with a proper nights rest.*
- **Lightly Active Hours steadily increased past the average nights rest** and then declined after 9 hours of sleep.
This trend line is much more in line with what I expected for the fairly active minutes chart.

- **Sedentary Hours steadily decreased as more sleep was enjoyed.**
  This generally reveals that users are likely to be less sedentary as more sleep is had. Strangely enough the sedentary minutes didn't sharply increase with more sleep (>10 hours), I'm going to understand this as a result of device functionality. Sleep may not equate to daily sedentary time which is why the decline in sedentary minutes is consistent regardless of how much of the day users are sleeping.

7. Recommendations

How can the marketing strategy be informed with the data presented?

- The data presents user trends that can be used to promote popular "healthy" habits. Bellabeat should identify ways to utilize this data and market their watch device "Time" alongside the app to drive current user engagement and promote new user sales.

Recommendation 1: Build reminder functions into Bellabeat Time/App.

- These reminders can promote healthy sleep habits (reminders to sleep at specific times) or promote activity during common hours (reminders to get active if sedentary minutes are being logged). Everyone leads a different lifestyle so the standard initial reminders can/should be based on the data trends here and refined overtime for individual user behaviors. This will be used for marketing but should also drive current user engagement (advertising revenue opportunity on the app!).

Recommendation 2: Market the reminder functions!

- Bellabeat's desire to grow their share of the smart device market within their target population (tech-familiar women) is reflected in the case study, the data presented, and also on their website. It's clear that the Bellabeat Time is designed to be a unique product that provides a fitness tracking device experience in a more timeless, classy package. Marketing the mechanical watch appearance while emphasizing these reminders alongside other fitness tracking functions (with the app) should capture the interest in potential buyers (driving sales figures within the target population).

Recommendation 3: Share sales data and previous marketing campaigns!

- This case study was an exercise using publicly available Fitbit data. In order to most effectively guide the marketing strategy at Bellabeat, having access to monthly/quarterly sales data and timelines of marketing campaigns would be most useful! This would allow analysts to explore previous campaigns that had positive impacts and ensure future experimental efforts are not duplicative efforts.